

%pip install -qU transformers accelerate datasets peft bitsandbytes

✓ 1. transformers

📌 Hugging Face company එකේ library එක.

👉 මෙතෙක් කරන්න පුලුවන්:

- BERT / GPT / T5 / Llama වගේ pre-trained models load කරගන්න
- Text generation, translation, summarization, Q&A වගේ NLP tasks
- Model training / fine-tuning

🚀 සරලව කියනව නම්:

“AI model use කරන්න, run කරන්න, train කරන්න” use කරන library එක.

✓ 2. accelerate

මේක Hugging Face library එකක්.

Model train කරනකොට GPU / TPU / CPU වලට model distribute කරලා fast training ලබාගන්න.

👉 Use cases:

- Multi-GPU training
- Mixed precision (FP16, BF16) training
- Training speed 10x quick වෙයි

🚀 Simple Sinhala:

“Training speed වැඩි කරන්න, large model several GPU වලට split කරන්න use කරන tool එක.”

✓ 3. datasets

දැන්තම large datasets download, load, process කරන්න fastest way.

👉 Use cases:

- Hugging Face datasets download
- CSV, JSON, Parquet, images handle කරන්න
- Training dataset split/clean/filter

🚀 Simple Sinhala:

“AI models train කරන්න දත්ත manage කරන්න හොඳම library එක – super fast data loader.”

✓ 4. peft

PEFT = Parameter Efficient Fine Tuning

මාඩ්‍රල් එකක parameters ගොඩක් update නොකර කොහොමද lightweight way එකක train කරන්නද කියලා help කරන library.

👉 Includes:

- LoRA
- Prefix Tuning
- P-Tuning
- Adapter methods

🚀 Simple Sinhala:

“Model full train කරන්නේ නැතුව, කොට්ටර කුඩා weight කිහිපයක් ආශ්‍රිත train කරලා cheap & fast fine-tuning කරන්න.”

✅ 5. bitsandbytes

මෙක 8-bit / 4-bit quantization library එක.

🔧 Main purpose:

- Large AI models low RAM වල run කරන්න
- GPU memory use 75%ක් පමණ කඩා බිඳා දමන්න
- Llama, Mistral වගේ models 4GB GPU එකකවත් run කරන්න

🚀 Simple Sinhala:

“Large AI model එකක් 8-bit/4-bit වලට compress කරලා low GPU usage වල run කරන්න.”

🔥 සාරාංශයක්

Library	Sinhala Meaning
transformers	AI models use/run/train කරන්න
accelerate	Training speed boost + multi-GPU support
datasets	Training data load/clean/manage super fast
peft	Lightweight fine-tuning (LoRA etc.)
bitsandbytes	Model quantization (8-bit / 4-bit)

```
import torch
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainingArguments,
    Trainer,
```

)

```
from peft import LoraConfig, get_peft_model, PeftModel
from datasets import Dataset, load_dataset
# Check device

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
if device != "cuda":
    print("WARNING: Running without a GPU may be very slow or may not fit a 7B model in memory.")
```

(1) import torch

import torch

👉 මොකක්ද මෙක?

PyTorch කියන්නේ machine learning models හදන්න use කරන library එක.

👉 සිංහලෙන්:

“AI model හදන්න, run කරන්න, GPU/CPU handle කරන්න use කරන main tool එක.”

(2) transformers libraries import කරගන්න

from transformers import (

AutoModelForCausalLM,

AutoTokenizer,

BitsAndBytesConfig,

TrainingArguments,

Trainer,

)

ඔබේ තියෙන එක එක items explain කරමු:

★ AutoModelForCausalLM

👉 මෙක මොකක්ද?

GPT, Llama, Mistral වගේ text generate කරන models load කරන්න.

👉 සිංහලෙන්:

“පෙර-පුහුණු කරපු (pretrained) text-producing AI model එකක් load කරන්න.”

★ AutoTokenizer

☛ මෙක මොකක්ද?

Text words → numbers (tokens) convert කරන්න.

AI modelsට letters/words තේරෙන්නේ නැත්නම් numbers (tokens) විතරයි.

☛ සිංහලෙන්:

“නෝට්පරයක් කියවලා, ඒක model එකට අර්ථවත් සංඛ්‍යාවලියක් (tokens) වෙලා යවන්න.”

★ BitsAndBytesConfig

☛ මෙක මොකක්ද?

Model එක 4-bit / 8-bit compress කරලා memory save කරන්න.

☛ සිංහලෙන්:

“Model එක කුඩා size එකකට compress කරලා low-memory GPU වල run කරන්න.”

★ TrainingArguments

☛ මෙක මොකක්ද?

Training එකේ settings set කරන්න:

- batch size
- learning rate
- epoch count
- save steps

☛ සිංහලෙන්:

“Model train කරන rules/setting තියන box එක.”

★ Trainer

☛ මෙක මොකක්ද?

Training process full එකම handle කරන machine.

☛ සිංහලෙන්:

“Model train කරන්න auto system එකක්.

ඔබ settings දීලා ‘train’ කියන command එක දෙන්නෙ විතරයි.”

🚀 (3) PEFT / LoRA import

```
from peft import LoraConfig, get_peft_model, PeftModel
```

★ PEFT (Parameter Efficient Fine-Tuning)

☛ Full model train කරන්න memory ගහනවා.

☛ LoRA = small layers train කරන light method.

☛ සිංහලෙන්:

“Model එකේ 100% train කරනවට අමතරව, model එකේ 1% වගේ small layers විතර නැවත train

කරන cheap හුඟක් fast method එක.”

LoraConfig

👉 LoRA setup:

- rank (r)
- dropout
- targets

get_peft_model

👉 original model එකට LoRA layers add කරනවා.

PeftModel

👉 trained LoRA model save/load කරන්න.

(4) Dataset import

from datasets import Dataset, load_dataset

👉 load_dataset — Hugging Face dataset load කරගන්න.

👉 Dataset — ඔබගේ text list එකකින් own dataset create කරන්න.

👉 සිංහලෙන්:

“AI model train කරන්න data load/handle කරන box එක.”

(5) device check

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
print("Using device:", device)
```

👉 computer එකේ GPU (NVIDIA CUDA) තියෙනවද බලනවා.

- GPU තියෙනවා → "cuda"
- GPU නෑ → "cpu"

👉 සිංහලෙන්:

“Computer එකේ AI models fast run කරන්න GPU තියෙනවද බලනවා.”

WARNING message

```
if device != "cuda":
```

```
    print("WARNING: Running without a GPU may be very slow or may not fit a 7B model in memory.")
```

👉 ඔබට GPU නැත්නම්

7B model (7 billion parameters) CPU වල run වෙන්නේ නැහැ, memory crash වෙනවා.

👉 සිහිපත්:

“GPU නැත්නම් 7B size model run කරන්නේ impossible almost. RAM overflow වෙයි. Slow ද? අතිශයම slow.”

```
model_name = "mistralai/Mistral-7B-v0.1" # Model selection

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)

# If padding token doesn't exist → assign eos token
if tokenizer.pad_token_id is None:
    print("Setting pad token to eos token")
    tokenizer.pad_token_id = tokenizer.eos_token_id

# Example sentences
inputs = tokenizer(["Hello", "How are you?"], padding=True, return_tensors="pt")

print(inputs)
```

Output idea:

input_ids:

```
[
  [101, 202, 2, 2],
  [ 55, 66, 77, 88]
]
```

model_name = "mistralai/Mistral-7B-v0.1"

මේ line එකෙන් කියනවා:

“හෙයි! මම HuggingFace වල තියෙන Mistral 7B model එක use කරනවා.”

Example:

```
model_name = "mistralai/Mistral-7B-v0.1"
```

➔ Why is this needed?

ඔයා later model load කරන්නෙ මේ නම use කරලා:

```
model = AutoModelForCausalLM.from_pretrained(model_name)
```

උදාහරණයක්:

- මොකක්ද මේ model?
 - Transformer language model
 - 7 Billion parameters
 - Text generation, chat, summarizing, instruction following

2. Tokenizer Load කිරීම

```
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
```

👉 Tokenizer කියන්නේ මොකක්ද?

Tokenizer = වචන → tokens (numbers) convert කරන machine.

Model එකට text කියවන්න බැහැ.

Model එකට නම් numbers (token IDs) කියවන්න පුළුවන්.

Example:

Text:

"Hello world"

Tokens:

[15496, 995]

👉 use_fast=True කියන්නේ?

- Rust වලින් ලියපු super fast tokenizer use කරන්න
- Long texts tokenize කරනකොට speed ×4 වගේ වැඩි

3. Check if pad token exists

if tokenizer.pad_token_id is None:

මේකෙන් check කරන්නේ:

“This tokenizer එකට padding token ID එක දාලා නැද්ද?”

හොඳටම simple කරන්නම්:

→ GPT-like models ("decoder-only") වල:

PAD token එක normally නැහැ.

එනිසා pad_token_id = None වෙලා එයි.

4. If pad_token not set → Set it to eos_token

```
print("Setting pad token to eos token")
```

```
tokenizer.pad_token_id = tokenizer.eos_token_id
```

👉 Why pad_token = eos_token ?

Because GPT-type models DON'T know what pad token is...

BUT

They DO know EOS token (</s>).

So easiest safe solution:

- Padding needed → give EOS token

🔥 Real Example for Understanding padding:

Imagine you want to batch 2 sentences:

Sentence 1: "Hello"

Sentence 2: "How are you?"

Tokens (assume):

"Hello" → [101, 202]

"How are you?" → [55, 66, 77, 88]

Batching requires equal length, so we must pad:

[101, 202, PAD, PAD]

[55, 66, 77, 88]

But GPT has no PAD token → model crashes ❌

So we use EOS instead:

PAD = EOS token

Example:

[101, 202, 2, 2] ← 2 = eos token id

[55, 66, 77, 88]

Now batching works perfectly.

🧠 Why padding is needed?

Because GPU wants:

- same shape
- same length
- same dimension

If sequences are different sizes →

model cannot batch them → training slow or error.

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True, # use double quantization for stability
    bnb_4bit_quant_type="nf4", # NormalFloat4, recommended 4-bit datatype
    bnb_4bit_compute_dtype=torch.bfloat16 # use bfloat16 for computations
)
# Load model with 4-bit quantization

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto", # let HF allocate layers across GPU (and CPU if needed)
)
model = model.to(device)
print("Model loaded.")
```

1. BitsAndBytesConfig කියන්නේ මොකක්ද?

BitsAndBytesConfig = HuggingFace / bitsandbytes library එක use කරලා

model එක 8-bit / 4-bit වලට compress (quantize) කරන්න use කරන configuration object එක.

➡ Quantization = model weight numbers small data types වලට convert කරන එක:

- 16-bit → 4-bit
- model size කප්පි වෙනවා
- RAM / VRAM usage අරක්කාවෙන් අඩුවෙනවා
- GPU එකට 7B, 13B වගේ models load කරන්න හැක

2. This whole config block meaning

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
```

)

මේකෙන් කියනවා:

“Model weights 4-bit වල load කරන්න සහ best performance/stability settings use කරන්න.”

අපි දැන් line by line බලමු 🙌

■ (1) load_in_4bit=True

👉 Model weight numbers (float16 / float32) 4-bit වලට convert කරලා load කරන්න.

Why?

- VRAM usage 3x-4x අඩුවෙයි
- Large models small GPUs වල run වෙයි (6GB / 8GB / 12GB GPU)

Example:

Normal model weight 16-bit (bfloat16):

Weight value: 0.238756567

4-bit quantized weight:

Weight value approx: 0.24 (4-bit)

**Accuracy a little reduced,
BUT memory usage dramatically reduced.**

■ (2) bnb_4bit_use_double_quant=True

👉 Double quantization enable කරනවා.

Simple Explanation:

Quantization = values compress කිරීම

Double quantization = compress කරපු values තවත් compress කිරීම (metadata quantize කිරීමත්).

Why use it?

- Model size even smaller
- VRAM usage 10-20% පමණ තවත් save වෙනවා
- FLOPS usage optimized

Example:

Without double quant:

- 7B model → ~5.2GB

With double quant:

- 7B model → ~4.3GB

(3) bnb_4bit_quant_type="nf4"

👉 Use NF4 (Normal Float 4) quantization type.

NF4 = ඕනම 4-bit method එකක් ඔස්සේ දී
Deep learning සඳහා accurate results.

Why NF4?

- mathematically designed to approximate real 16-bit weights
- best quality among all 4-bit quantizers
- recommended by Meta + HuggingFace + Mistral

Example:

Binary 4-bit numbers vs NF4:

- FP4 = simple linear mapping (not accurate)
- NF4 = distribution-aware → far more accurate
- Less loss in performance

So this is the BEST 4-bit format.

(4) bnb_4bit_compute_dtype=torch.bfloat16

👉 Even though weights are 4-bit,
actual math operations (matrix multiply, attention)
compute in bfloat16.

Why?

- bfloat16 = extremely stable & fast
- more robust than FP16
- supports wider range of values
- good for both training + inference

Simple Analogy:

- Store data cheap (4-bit)
- But calculations high-quality (bfloat16)

This is why 4-bit quantized models still generate high-quality text.

3. Load Model with Quantization

```
model = AutoModelForCausalLM.from_pretrained(  
    model_name,
```

```
quantization_config=bnb_config,  
  
device_map="auto"  
  
)
```

Here's what's happening:

→ **quantization_config=bnb_config**

Apply everything you defined earlier.

→ **device_map="auto"**

HuggingFace automatically decides:

- which model layer goes to GPU
- which layer goes to CPU
- depending on available VRAM

If GPU is small:

- 70% layers on GPU
- 30% on CPU
- Still works smoothly.

This avoids "CUDA out of memory".

4. Move model to device (GPU/CPU)

```
model = model.to(device)
```

If device = "cuda" → model goes to GPU

If device = "cpu" → model on CPU

5. Print message

```
print("Model loaded.")
```

FULL SINHALA SUMMARY

- load_in_4bit=True → model weights 4-bit වලට compress කරලා memory save කරන්න
- double quant → memory තවත් save + stability maintain
- nf4 → best 4-bit datatype for LLMs
- compute dtype = bfloat16 → 4-bit weights use කරලා high-quality calculations
- device_map="auto" → GPU/CPU mix-use කරන්න automatically
- model.to(device) → model load කරන්න correct device එකට

```
# Quick tokenizer and padding test
```

```
texts = ["Hello world!", "This is a slightly longer sentence."]
enc = tokenizer (
    texts,
    padding=True,
    truncation=True,
    max_length=10,
    return_tensors="pt"
)
print("Token IDs:\n", enc["input_ids"])
print("Attention masks:\n", enc["attention_mask"])
print("Pad token id:", tokenizer.pad_token_id)
```

1. Code block overview

```
texts = ["Hello world!", "This is a slightly longer sentence."]
```

```
enc = tokenizer(texts, padding=True, truncation=True, max_length=10, return_tensors="pt")
```

මේදී කිරවෙන්නේ:

- string දෙක tokenize කිරීම
- padding use කරලා **same length** කරන්න
- PyTorch tensors return කරන්න

මෙම test එකෙන් check කරනවා:

- ✓ pad token set properlyද?
- ✓ padding වලින් token IDs equal lengthද?
- ✓ attention mask හරියට generate වෙද්ද?

Step-by-step Sinhala Explanation

1. texts = [...]

We have two sentences:

1. "Hello world!"
2. "This is a slightly longer sentence."

වෙනස් length තියෙන text දෙකක් purposely use කරනවා
→ padding check කරන්න.

2. Tokenizer call

```
enc = tokenizer(
```

```
    texts,
```

```
    padding=True,
```

```
truncation=True, #truncation = cutting
max_length=10,
return_tensors="pt"
)
```

දැන් සෑම argument එකම explain කරලා දෙන්නම් 📌

◆ padding=True

👉 All sentences must be **same token length**

if shorter → add PAD token

if longer → truncate

Example:

Text	Token length
"Hello world!"	3 tokens
"This is a slightly longer sentence."	8 tokens

If max_length=10 → both outputs become **10 tokens**.

Shorter sequences padded like this:

[101, 2312, 999, 2, 2, 2, 2, 2, 2]

↑ pad tokens (2 = eos/pad)

◆ truncation=True (cutting)

👉 If a sentence becomes **longer than max_length**, excess tokens are cut.

Example:

If encoded tokens = 15

max_length = 10

→ last 5 tokens removed.

◆ max_length=10

Force every output sequence to length = **10 tokens**.

Without this, padding will match longest sequence,
but here we force a fixed size.

◆ return_tensors="pt"

👉 Return PyTorch tensors (instead of Python lists)

Output will look like:

```
tensor([
  [101, 202, 203, 2, 2, 2, 2, 2, 2],
  [101, 44, 55, 66, 77, 88, 99, 123, 2, 2]
])
```

This is what models expect.

(

Python list vs PyTorch tensor

◆ Python list (normal Python data)

```
[101, 202, 203, 2, 2, 2]
```

This is just a **list**.

Python can use it, but **GPU cannot use it**.

◆ PyTorch tensor (GPU-friendly numeric array)

```
tensor([101, 202, 203, 2, 2, 2])
```

This is a **tensor**, which is like a super-optimized array.

Models (neural networks) can only read **tensors**, not Python lists.

■ 2. Why does the model expect a tensor?

LLM models work on **GPU / CPU mathematical operations**.

They need:

- matrix multiplication
- attention layers
- vector operations
- optimized CUDA kernels

👉 **These operations cannot run on normal Python lists.**

They must run on **tensors**, because tensors:

- store numbers in GPU memory
- run extremely fast
- use PyTorch operations
- support automatic differentiation
- support batching

)

3. Print Output

```
print("Token IDs:\n", enc["input_ids"])
```

```
print("Attention masks:\n", enc["attention_mask"])
```

```
print("Pad token id:", tokenizer.pad_token_id)
```

Let's explain each 🙋

1. input_ids

This is the tokenized version of your texts.

Example:

```
tensor([
  [15496, 995, 2, 2, 2, 2, 2, 2, 2], ← padded output
  [566, 314, 257, 4543, 3852, 1842, 13, 2, 2, 2]
])
```

■ 2 = pad token (because earlier we set pad = eos)

2. attention_mask

This tells the model which positions it should **pay attention to**.

Rules:

- 1 → real token (not padded)
- 0 → padding token (ignore)

Example:

```
tensor([
  [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
  [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]
])
```


])

This keeps the model from learning garbage from padding.

3. tokenizer.pad_token_id

This prints which token used as padding.

If you previously set:

```
tokenizer.pad_token_id = tokenizer.eos_token_id
```

Then output may be:

Pad token id: 2

This confirms that **pad token = eos token** successfully set.

Why this test is important?

This test checks:

- ✓ padding works
- ✓ attention mask works
- ✓ max_length truncation correct
- ✓ tokenizer.pad_token_id is set properly
- ✓ model training/inference won't crash later

If this test fails →

LLM training will break instantly.

Super Simple Sinhala Summary

- මේ code එක text දෙක tokenize කරනවා
- padding=True → දෙකම same length කරන්න PAD tokens දානවා
- truncation=True → max_length exceed වුනොත් කැපු දානවා
- return_tensors="pt" → torch tensors දෙනවා
- input_ids → token numbers
- attention_mask → pad 0 / real tokens 1
- pad_token_id → pad token check

```
# Load the financial QA dataset from Hugging Face
```

```
raw_qa = load_dataset("OpenFinAL/Financial_Question_Answering", split="train")
```

```
print("Raw financial QA split size:", len(raw_qa))
```

```
print("Example entry:", raw_qa[0])
```

 මෙහෙම මොකක්ද වෙන්නේ?

✓ 1. Dataset එක Hugging Face වෙබ්සයිට් එකෙන් load කරනවා

load_dataset() කියන්නේ Hugging Face datasets library එකේ function එකක්. එකෙන් online තියෙන datasets local machine එකට load කරගන්න පුළුවන.

✓ 2. Dataset එකේ sample count print කරනවා

len(raw_qa) → dataset එකේ total items කීයද කියලා බලාගෙන print කරනවා.

✓ 3. First example එක print කරනවා

raw_qa[0] → dataset එකේ පළමුවන QA example එක.

🔍 Code එක

```
# Load the financial QA dataset from Hugging Face
```

```
raw_qa = load_dataset("OpenFinAL/Financial_Question_Answering", split="train")
```

```
print("Raw financial QA split size:", len(raw_qa))
```

```
print("Example entry:", raw_qa[0])
```

👤 විස්තරාත්මක සිංහල Explanation

■ 1. Dataset එක Load කිරීම

```
raw_qa = load_dataset("OpenFinAL/Financial_Question_Answering", split="train")
```

මේ line එකෙන්:

- "OpenFinAL/Financial_Question_Answering" කියන dataset එක HuggingFace hub එකෙන් ගන්නවා.
- split="train" කියන්නේ "training dataset portion" එක load කරන්න කියන එක.

🔍 Example idea

ඔබ්බෙකු AI model train කරන්න financial questions කිහිපයක් හදලා HuggingFace එකේ දාලා තියෙනවා කියලා හිතුවොත් –

ඔබ මෙහෙම load_dataset() call එකක් දාලා ඒ data ඔයාගේ Python code එකට ගන්නවා.

■ 2. Dataset size එක print කිරීම

```
print("Raw financial QA split size:", len(raw_qa))
```

len(raw_qa) → dataset එකේ **entries කොච්චරද** කියලා කියනවා.

උදා:

If dataset size = 10,000

→ "Raw financial QA split size: 10000" කියලා print වෙයි.

■ 3. Example එක print කිරීම

```
print("Example entry:", raw_qa[0])
```

මෙන්න dataset එකේ පළමු **example** එක print කරනවා:

උදා: (මෙහෙම වගේ format එකක් එයි)

```
{
  'question': "What is a stock split?",
  'answer': "A stock split is when a company divides existing shares to increase liquidity.",
  'category': "stocks"
}
```

```
# The raw dataset has columns 'Question' and 'Answer'.
```

```
# Rename them to lowercase for convenience.
```

```
dataset = raw_qa.rename_columns({"Question": "question", "Answer": "answer"})
print("Total examples in dataset:", len(dataset))
print("Example 1:", {k: dataset[0][k] for k in ["question", "answer"]})
```

```
# Split into train and eval splits for formal evaluation
```

```
splits = dataset.train_test_split(test_size=0.1, seed=42)
train_dataset = splits["train"]
eval_dataset = splits["test"]
print("Train size:", len(train_dataset))
print("Eval size:", len(eval_dataset))
```

```
# We'll reuse an eval example later for qualitative before/after comparison
eval_example = eval_dataset[0]
```

1. Columns rename කිරීම

```
dataset = raw_qa.rename_columns({"Question": "question", "Answer": "answer"})
```

✓ මෙකෙන්න:

- “Question” → “question”
- “Answer” → “answer”

Why rename?

LLM training කරනවා නම් column names **lowercase + short** තිබ්බොත් easy.

Example:

ඔබට Excel sheet එකක් තියෙනවා කියලා හිතන්න.

Original:

- **Question**
- **Answer**

AI training කාලේ:

- **question**
- **answer**

👉 Lowercase names coding time එකේ පහසුව.

2. Print dataset summary + first example

```
print("Total examples in dataset:", len(dataset))
```

```
print("Example 1:", {k: dataset[0][k] for k in ["question", "answer"]})
```

✓ `len(dataset)`

Dataset එකේ total Q/A pairs count.

✓ `{dataset[0][k] ... }`

පළමු row එකෙන් **question + answer** මட்டුම print කරනවා.

🧑 Example output (imaginary):

Total examples: 5000

Example 1: {

'question': "What is net income?",

'answer': "Net income is revenue minus expenses."

}

3. Train/Test (Evaluation) split කිරීම

```
splits = dataset.train_test_split(test_size=0.1, seed=42)
```

```
train_dataset = splits["train"]
```

```
eval_dataset = splits["test"]
```

✓ `test_size=0.1`

10% data → evaluation

90% data → training

✓ seed=42

Randomness fix කරලා same split repeat වෙනවා (deterministic)



Simple Sinhala Example

Imagine you have **1000 financial questions**.

You want to teach a model:

- 900 questions → **training**
- 100 questions → **testing (eval)**

AI model එක train කරද්දී **train data** නං answers “learning” කරනවා.

Eval data එක නම් model එක test කරනවා, “ඔයාලා කියපු දේ පොහොසත් ද?” කියලා.

4. Train / Eval sizes print කරනවා

```
print("Train size:", len(train_dataset))
```

```
print("Eval size:", len(eval_dataset))
```

Example:

Train size: 4500

Eval size: 500

5. Evaluation example එක future use එකට save කරනවා

```
eval_example = eval_dataset[0]
```

✓ This means:

Eval dataset එකේ පළමු question-answer pair එක future comparison එකකට save කරනවා.

For example:

LLM fine-tune කිරීමට කලින්:

- Model answer → A

Training කල පස්සේ:

- Model answer → B

ඔය දෙකක් compare කරන්න මේ example එක use කරනවා.

```
MAX_LEN = 256
```

```
def preprocess_function(example):
```

```
    """Format a simple QA prompt and tokenize it for causal LM fine-tuning.
```

```
    We use the pattern: "Question: <question>\nAnswer: <answer>" and train the
```

```
    model to predict every token in this sequence (standard next-token
```

```
    prediction objective).
```

```
    """
```

```
    question = example["question"]
```

```
    answer = example["answer"]
```

```
    prompt = f"Question: {question}\nAnswer:"
```

```
    full_text = prompt + " " + answer + tokenizer.eos_token
```

```
    enc = tokenizer(
```

```
        full_text,
```

```
        truncation=True,
```

```
        padding="max_length",
```

```
        max_length=MAX_LEN,
```

```
    )
```

```
    enc["labels"] = enc["input_ids"].copy()
```

```
    return enc
```

```
# Apply preprocessing to train and eval splits separately
```

```
tokenized_train = train_dataset.map(preprocess_function, remove_columns=train_dataset.column_names)
```

```
tokenized_eval = eval_dataset.map(preprocess_function, remove_columns=eval_dataset.column_names)
```

```
# Set the dataset format for PyTorch
```

```
tokenized_train.set_format("torch")
```

```
tokenized_eval.set_format("torch")
```

```
print("Tokenized train fields:", tokenized_train.column_names)
```

```
print("Tokenized eval fields:", tokenized_eval.column_names)
```

```
print("Example tokenized train input_ids (first 20 tokens):", tokenized_train[0]["input_ids"][:20])
```

ඔබ AI model එකට කියනව:

"මියාට තියෙන Question + Answer book එකක් tokenize කරන්න."

එක එක page එකට:

- Question
- Answer

මොනම හරි model එකට train වෙන format එකක් හදලා දෙන්න.

ඒ format එක:

Question: <question>

Answer: <answer> <EOS>

● STEP 1 — MAX_LEN = 256

256 කියන්නේ:

- එක් sample එක tokenize කරලා දෙන max length.
- Text 256 tokens (words/word pieces) ඉක්මවුනොත් ඝਟ කරලා දෙනවා.
- Length 256 ට අඩුනම් zeros (padding) දාලා 256 කරන්න උදව් කරනවා.

කිව්වොත්:

"සෑම paper එකක්ම page size එක 256-word limit එකක්."

● STEP 2 — preprocess function එක

Now let's break it EXTREMELY SIMPLE.

✓ Inside preprocess_function()

(A) Get question and answer

```
question = example["question"]
```

```
answer = example["answer"]
```

ධාරන කරන්න example එක මෙහෙමයි:

```
{  
  
  "question": "What is income tax?",  
  
  "answer": "Income tax is a tax you pay on your earnings."  
  
}
```

තමන් එකෙන් question + answer string දෙක ගන්නවා.

✓ (B) Create the training text (full_text)

prompt = f"Question: {question}\nAnswer:"

full_text = prompt + " " + answer + tokenizer.eos_token

අපි කියන්නේ:

- Model එකට Question එක print කරන්න
- Then model should predict the answer

හදලා තියෙන final text:

Question: What is income tax?

Answer: Income tax is a tax you pay on your earnings. <eos>

මෙකම model එකට feed වෙන full_text.

✓ (C) Tokenize full_text

```
enc = tokenizer(full_text,  
                 truncation=True,  
                 padding="max_length",  
                 max_length=MAX_LEN)
```

Tokenizing = convert text → numbers

For example:

"Question:" → 1321

"What" → 212

"is" → 98

"income" → 4012

"tax?" → 5523

So entire sentence becomes something like:

[1321, 212, 98, 4012, 5523, 1367, 6423, 13, 55, 102, ... padded zeros ...]

✓ (D) Create labels (Super Important)

```
enc["labels"] = enc["input_ids"].copy()
```

ඒ කියන්නේ:

■ input_ids (model input)

■ labels (correct answer)

`copy()` කරන්නේ `input_ids` `copy` එකක්.

Why?

Because causal LMs train with “next token prediction”:

Model sees:

Question: What is income tax?

Answer:

Model tries to predict next token:

Income

Then next token:

tax

Then next token:

is

... etc.

So labels = `input_ids` (shifted internally).

කෙටි version:

Model learns to generate the answer.

✓ (E) Return the tokenized dictionary

This dictionary contains:

- `input_ids`
- `attention_mask`
- `labels`

Model training ට ඔබ කාලි.

● STEP 3 — Apply to entire dataset

```
tokenized_train = train_dataset.map(preprocess_function)
```

```
tokenized_eval = eval_dataset.map(preprocess_function)
```

`map` = every row → preprocess

● STEP 4 — Convert dataset into PyTorch tensors

```
tokenized_train.set_format("torch")
```

```
tokenized_eval.set_format("torch")
```

Now everything becomes torch tensors:

```
tensor([1321, 212, 98, ...])
```

● STEP 5 – Print sample

```
print(tokenized_train[0]["input_ids"][:20])
```

It will show first 20 tokens like:

```
tensor([1321, 212, 98, 4012, 5523, 1367, 6423, ...])
```

```
# Configure LoRA
```

```
lora_config = LoraConfig(
```

```
    r=16,
```

```
    lora_alpha=32,
```

```
    target_modules=["q_proj", "v_proj"],
```

```
    lora_dropout=0.05,
```

```
    bias="none", # do not update bias terms
```

```
    task_type="CAUSAL_LM",
```

```
)
```

```
# Wrap the model with LoRA adapters
```

```
model = get_peft_model(model, lora_config)
```

```
# Print trainable vs total parameters
```

```
model.print_trainable_parameters()
```

✅ LoRA කියන්නේ මොකක්ද? (පළවෙනි තේරුම)

LoRA = Low-Rank Adaptation

👉 ඉතා විශාල AI model එකක් (LLM එකක්) update කරන්න හිතුවෙන්

සම්පූර්ණ model parameters update නොකර,

model එකේ කොහොමද සලකනවා? කියන ඝාගය (Attention matrices) පොඩි adapter layers දෙකක් එක් කරලා update කරන technique එක.

🔴 ඉතාම සරල උදාහරණයක්:

- Suppose 100kg එකක් උස්සන machine එකක් තියනවා (LLM model).

- ඔයාට machine එකට හොඳම balance එකක් දීලා තවත් හරි හැටි වැඩ කරන්න දෙන්න ඕන.
- Machine එක සම්පූර්ණයෙන්ම repaire එකක් කරන්න ඕන නෑ.
- පොඩි two small parts එකතු කරලා machine එක adjust කරන්න පුළුවන්.

ඒ parts දෙක තමයි LoRA adapters.

1) LoraConfig() – LoRA adapter එක set කරන config එක

```
lora_config = LoraConfig(
```

```
    r=16,
```

```
    lora_alpha=32,
```

```
    target_modules=["q_proj", "v_proj"],
```

```
    lora_dropout=0.05,
```

```
    bias="none",
```

```
    task_type="CAUSAL_LM",
```

```
)
```

මේකේ value වල අර්ථය:

◆ $r = 16$

👉 මේක LoRA adapter එකේ rank.

Super simple meaning:

🔴 Adapter layer එකේ size එක කොච්චරද? කියන එක.

Rank එක වැඩි වුනාම model update එක තව powerful.

Rank එක අඩු වෙද්දි update එක cheap.

✓ Example:

- $r = 2 \rightarrow$ small adapter, cheap, low power
- $r = 16 \rightarrow$ medium power
- $r = 64 \rightarrow$ very powerful, expensive

ඔයා $r=16$ දාලා තියෙන්නේ good balance.

◆ $\text{lora_alpha} = 32$

👉 Adapter update එක කියවන ගුණකය (scaling factor)

Simple meaning:

🔴 LoRA adapter එක model එකේ output එකට කීයද බලපෑවැන්නේ?

High alpha = strong effect

Low alpha = weak effect

◆ `target_modules = ["q_proj", "v_proj"]`

මේක IMPORTANT.

Transformers model එකේ attention mechanism තියනවා.

එකේ parts:

- `q_proj` = Query projection
- `v_proj` = Value projection

LoRA කියන්නේ මේ two parts වලට adapters දානවා.

✓ Simple Example:

Attention = කාට වගේද?

→ AI එක text එකකින් "මොනාටද ජ්‍යෙෂ්ඨ උපදේශක" කියලා filter කරන brain part එක.

LoRA adapters ඒ brain part එකේ හොඳි adjusters.

◆ `lora_dropout = 0.05`

👉 Training එකේ dropout percentage (regularization).

Overfitting අවුල් එන්න දෙන්නේ නැතුව.

◆ `bias = "none"`

👉 Bias values update කරන්නේ නෑ.

Only LoRA adapters update වෙනවා.

◆ `task_type = "CAUSAL_LM"`

👉 Model එක text generate කරන model (LLM) කියලා කියනවා.

● 2) `get_peft_model()`

`model = get_peft_model(model, lora_config)`

🔴 මේ line එක Model එකට LoRA adapters attach කරනවා.

දිලීර example:

Original Model = big truck

LoRA adapters = two small helper engines

`get_peft_model()` = අර්බුද helper engines truck එකට bolt කරලා attach කරන tool එක.

● 3) `model.print_trainable_parameters()`

`model.print_trainable_parameters()`

👉 මේක print කරන්නේ:

- Total parameters (example: 7 billion)
- Trainable parameters (example: 2 million only)
- Percentage (example: 0.03% only trainable)

📌 LoRA එකේ idea එකම මෙහෙමයි:

Big model එක freeze—change කරන්නෙ නෑ

Only tiny adapter layers update—cheap + fast

LoRA කියන්නේ මොකක්ද? (සර්ව සාමාන්ය explanation)

Imagine **AI model එකක්** කියන්නේ:

→ වැඩිම **memory** තියෙන **10,000-page** පුරුද්දක්

(LLM model = billions parameters)

ඔයාට ඒ 10,000 පොත් සම්පූර්ණයෙන්ම *rewrite* කරන්න cost එකත්, time එකත් නෑ.

! LoRA: "පොත முழுவதும் rewrite නොකර,

එක පොතට එක්කන්න **පොඩි add-on notes pages** දෙකක්".

Training time →

AI model එකේ original pages *change* වෙනවා නෑ.

Update වෙන්නේ LoRA add-on pages (tiny notes) විතරයි.

දැන් ඔයා දුන්න **code** එක **EASY WAY** එකට explain කරමු

■ PART 1 — LoraConfig()

```
lora_config = LoraConfig(
```

```
    r=16,
```

```
    lora_alpha=32,
```

```
    target_modules=["q_proj", "v_proj"],
```

```
    lora_dropout=0.05,
```

```
    bias="none",
```

```
    task_type="CAUSAL_LM",
```

```
)
```

අපි මේක කොටස් කේ.

■ 1) **r = 16** → Add-on page size

Think:

- Original model = Big textbook
- LoRA adapter = small note paper

r = 16 කියන්නේ:

"Note paper එකේ size/strength එක".

Small r = small note (less learning)

Large r = bigger note (more learning)

■ 2) **lora_alpha = 32** → Note paper එක model එකට effect වෙන මුලු බලය

Imagine:

- Note paper එක "අඩු effect" නම් model එකට හුඟාක් change නෑ.
- Note paper එක "අධික effect" නම් model එකට big impact.

lora_alpha = "note paper influence power".

■ 3) **target_modules=["q_proj", "v_proj"]**

👉 adapters attach වෙන්නේ කෙලින්ම AI brain එකේ two parts වලට:

🧠 **q_proj** = "මොන words වලට focus කරන්නද?"

🧠 **v_proj** = "focus කරන words වල meaning ගන්නද?"

LoRA says:

"හරි, model එකේ focus & meaning parts පෙන්නපන්.
ඒ දෙකට add-on notes attach කරන්න."

That's it.

■ 4) **lora_dropout = 0.05**

Training හිඟලා යන්නේ නැතුව

පොඩි random drop feature 5% ක්.

Like:

"Notes 100% trust කරන්න එපා, randomව 5% ignore කරලා learn කරන්න."

■ 5) **bias="none"**

Model original bias parts update නොකරන්න කියන එක.

6) task_type="CAUSAL_LM"

Model එක type:

"Text generate කරන LLM model එකක්."

PART 2 – Wrap the model

```
model = get_peft_model(model, lora_config)
```

Super simple meaning:

Original model + LoRA add-on note pages = New upgraded model

This line = “Adapters model එකට connect කරන්න.”

PART 3 – Parameter report

```
model.print_trainable_parameters()
```

This prints:

Type	Count
Total model parameters	7,000,000,000
Trainable (LoRA only)	2,000,000

Meaning:

👉 **7 billion** parameters freeze

👉 **Just tiny 2 million** update

LoRA =

fast training + cheap GPU + same accuracy.

☀️ NOW THE BEST REAL LIFE EXAMPLE (ඔයාට SET වෙන්න)**

Imagine:

🎓 You are a student

You have **big textbook** = original LLM

You have **small notebook** = LoRA adapters

Teacher කියනවා:

“Don’t rewrite your textbook.

Write only the important points in your small notebook.”

Exam time:

- Textbook = freeze
- Notebook = updated

You answer exam using:

- textbook knowledge
- plus notebook extra learnings

This is EXACT LoRA.

```
# Baseline generation before fine-tuning using one financial QA example
```

```
eval_question = eval_example["question"]
prompt = f"Question: {eval_question}\nAnswer:"
model.eval()
inputs = tokenizer(prompt, return_tensors="pt").to(device)
with torch.no_grad():
    outputs = model.generate(**inputs, max_new_tokens=64, do_sample=False)
base_answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("Model output before fine-tuning on this example:")
print(base_answer)
```

Purpose of this code

Before we **fine-tune** the model with a specific dataset, we want to see **how the model behaves on this example “by default”**.

- This is called **baseline generation**.
- Basically: “Before I teach it, let’s see what it answers on its own.”

Step 1 — Get the question

```
eval_question = eval_example["question"]
```

- eval_example = your dataset example (a dictionary)
- "question" = the question we want the model to answer

Example:

```
eval_example = {"question": "What is the profit of company X in 2024?"}
```

```
eval_question = eval_example["question"]
```


Now, eval_question = "What is the profit of company X in 2024?"

■ Step 2 — Make prompt

```
prompt = f"Question: {eval_question}\nAnswer:"
```

- We format the question into a **prompt** that the model can understand.
- `f""` = f-string in Python (replace {eval_question} with actual question)

Result:

```
prompt = "Question: What is the profit of company X in 2024?\nAnswer:"
```

- `\n` = new line
- The model now sees: “Here is a question. Generate the answer.”

■ Step 3 — Set model to evaluation mode

```
model.eval()
```

- This tells PyTorch:

“Don’t calculate gradients. Don’t train. Only run inference.”

- Why? Because we are **just generating output**, not training.

■ Step 4 — Tokenize the prompt

```
inputs = tokenizer(prompt, return_tensors="pt").to(device)
```

- `tokenizer` = converts text into **numbers** (model understands numbers, not letters)
- `return_tensors="pt"` = return as PyTorch tensors
- `.to(device)` = move tensors to GPU (or CPU)

Example:

```
prompt = "Question: What is 2+2?\nAnswer:"
```

```
inputs = tokenizer(prompt, return_tensors="pt")
```

```
print(inputs)
```

Output might look like:

```
{'input_ids': tensor([[101, 2000, 2023, 102]]), 'attention_mask': tensor([[1,1,1,1]])}
```

- These numbers represent words.

■ Step 5 — Generate the model output

```
with torch.no_grad():
```

```
outputs = model.generate(**inputs, max_new_tokens=64, do_sample=False)
```

- `torch.no_grad()` = don't compute gradients (we are **just running the model**)
- `model.generate()` = model generates output **based on prompt**
- `max_new_tokens=64` = model can generate **up to 64 words/tokens**
- `do_sample=False` = model will generate **deterministic output** (no randomness)

Important: This is **baseline**, so we want to see **default model behavior**, not random guesses.

■ Step 6 – Decode output back to text

```
base_answer = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

- `outputs[0]` = generated numbers
- `tokenizer.decode()` = convert numbers back to human-readable text
- `skip_special_tokens=True` = remove extra tokens like <pad> or <eos>

■ Step 7 – Print the answer

```
print("Model output before fine-tuning on this example:")
```

```
print(base_answer)
```

Example output:

Model output before fine-tuning on this example:

The profit of company X in 2024 is not publicly available.

- This is the **baseline answer**.
- Later, after **fine-tuning**, the model should answer **more accurately**.

```
model.enable_input_require_grads()
```

```
model.gradient_checkpointing_enable()
```

```
training_args = TrainingArguments(
```

```
    output_dir="outputs",
```

```
    overwrite_output_dir=True,
```

```
    per_device_train_batch_size=1, # smaller per-device batch to fit 7B on Colab
```

```
    per_device_eval_batch_size=1, # smaller eval batch to avoid OOM during eval
```

```
    gradient_accumulation_steps=16, # keep a decent effective batch size
```

```
    num_train_epochs=2, # fewer epochs to keep runtime and memory manageable
```

```
    learning_rate=1e-4, # stable LR for LoRA/QLoRA on 7B
```

```
    weight_decay=0.01, # light regularization to help generalization
```

```
    warmup_ratio=0.1, # warmup to avoid early instability
```

```
    fp16=True,
```

```
    gradient_checkpointing=True, # save memory at the cost of some compute
```

```
    logging_steps=1,
```

```
    logging_first_step=True,
```

```

eval_strategy="steps", # run evaluation every N steps
eval_steps=25, # evaluate less frequently to reduce overhead
optim="paged_adamw_8bit", # 8-bit Adam optimizer (bitsandbytes)
)
trainer = Trainer(
    model=model,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_eval,
    args=training_args,
)
print("Starting training...")
trainer.train()

print("Training done.")# Run a final evaluation pass to see eval loss
metrics = trainer.evaluate()
print("Eval metrics:", metrics)

```

■ Step 1 – Enable input grads & gradient checkpointing

```
model.enable_input_require_grads()
```

```
model.gradient_checkpointing_enable()
```

```
enable_input_require_grads()
```

- Normally, **original model frozen** → gradients don't compute
- LoRA training එකේ, **inputs gradients compute කරන්න** කියන එක

Example:

- Notebook page = trainable
- But we need **brain (original model) input to flow gradient** to update notebook correctly

```
gradient_checkpointing_enable()
```

- Memory save කරන්න use කරන technique
- **Idea:** intermediate results (activations) store නොකරලා **on-demand recompute** කරනවා
- Memory reduce කරයි, compute time ටික වැඩේ

Example:

- Normally, “take a photo of every step” memory = huge
- Checkpointing = “remember only some key steps, recompute rest” → save memory

■ Step 2 – TrainingArguments

```

training_args = TrainingArguments(
    output_dir="outputs",
    overwrite_output_dir=True,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=16,
    num_train_epochs=2,
    learning_rate=1e-4,
    weight_decay=0.01,
    warmup_ratio=0.1,
    fp16=True,
    gradient_checkpointing=True,
    logging_steps=1,
    logging_first_step=True,
    eval_strategy="steps",
    eval_steps=25,
    optim="paged_adamw_8bit",
)

```

Explain line by line:

◆ 1) output_dir="outputs"

- Train result save කරන folder
- Model + LoRA adapter files saved here

◆ 2) overwrite_output_dir=True

- Folder already තියෙනවා නම් **replace කරලා write කරන්න** කියන එක

◆ 3) per_device_train_batch_size=1 & per_device_eval_batch_size=1

- Batch size = එකවර GPU එකට දෙන examples number
- 7B model (huge) → small batch (1) to avoid **OOM (out of memory)**

Example:

- Big bag + small box = 1 item at a time fits perfectly
- ◆ 4) `gradient_accumulation_steps=16`
- Batch size 1 → 16 steps accumulate කරන්න → effectively batch = 16
 - GPU memory limit workaround
- ◆ 5) `num_train_epochs=2`
- Training = 2 passes over whole dataset
 - Small dataset / Colab friendly
- ◆ 6) `learning_rate=1e-4`
- Step size for LoRA training
 - Small enough = stable learning
- ◆ 7) `weight_decay=0.01`
- Light regularization → prevent overfitting
 - Example: don't let notebook pages memorize wrong things too hard
- ◆ 8) `warmup_ratio=0.1`
- Learning rate **gradually increase first 10% steps** → avoid early instability
- ◆ 9) `fp16=True`
- Half-precision → memory save + faster training
- ◆ 10) `gradient_checkpointing=True`
- Already explained above → save memory
- ◆ 11) `logging_steps=1 & logging_first_step=True`
- Show training info **every step**
 - First step logging enabled
- ◆ 12) `eval_strategy="steps" & eval_steps=25`
- Evaluate model **every 25 steps**
 - Check loss on eval dataset
 - Reduce overhead → don't evaluate too often
- ◆ 13) `optim="paged_adamw_8bit"`
- Optimizer = AdamW in **8-bit precision**
 - Memory efficient
 - Works very well with LoRA / QLoRA

Step 3 — Trainer setup

```
trainer = Trainer(  
  
    model=model,  
  
    train_dataset=tokenized_train,  
  
    eval_dataset=tokenized_eval,  
  
    args=training_args,  
  
)
```

- model = LoRA attached model
- train_dataset = tokenized training examples
- eval_dataset = tokenized validation examples
- args = TrainingArguments from above

Trainer = high-level PyTorch wrapper → handles loops, optimizer, gradient accumulation, logging automatically.

Step 4 — Start training

```
print("Starting training...")
```

```
trainer.train()
```

```
print("Training done.")
```

- trainer.train() → starts fine-tuning process
- LoRA adapters update → original model frozen

Step 5 — Final evaluation

```
metrics = trainer.evaluate()
```

```
print("Eval metrics:", metrics)
```

- After training, evaluate performance
- Metrics = **loss / accuracy / other evaluation scores**
- Lets you know **how well LoRA adapters learned**

Super simple analogy

Imagine:

- **Big textbook** = frozen GPT model
- **Small notebook pages** = LoRA adapters
- **Trainer + TrainingArguments** = teacher giving instructions:
 - How fast to write (learning_rate)
 - How many pages per session (batch_size)
 - When to check homework (eval_steps)

- Save memory tricks (fp16, gradient_checkpointing)
- After training → notebook pages updated → **model improved for your dataset**